Homework 5 Report
Wilson Neira

Part I: Reading!

Chapter 2 Reading of Distributed Systems for Fun and Profit

Updated 3 days ago by Wilson Neira

For me, what I found most informative was how this chapter 2 explained the tradeoffs between consistency, availability, and partition tolerance in a very practical way, especially how strong consistency increases latency and limits availability, which relates to real cloud systems. I was familiar with working across different abstraction layers and dealing with distributed components from my work at DTCC, where I automated AWS Aurora database version checks across readers, writers, regions, and global clusters using CloudBees Jenkins, so I've sort of seen how different nodes can drift and why coordination and failure assumptions matter in real systems.

week_5_readings

Chapter 2 Reading of Distributed Systems for Fun and Profit

Updated 3 days ago by Nithish Kadam Ganesh

Chapter 2 most informative in how it reframed distributed systems as a series of abstraction choices rather than just implementation techniques. The discussion about system models, unreliable networks, lack of synchronized clocks, and independent node failures made it clear that many "guarantees" in distributed systems are only valid within specific assumptions. I particularly appreciated how CAP and FLP were presented not as trivia to memorize, but as logical consequences of those assumptions. That shift in perspective makes you question what your system is truly promising versus what is simply an abstraction layered over messy reality.

I was already familiar with CAP tradeoffs, eventual consistency, and replication concepts from my distributed systems coursework and from building distributed microservices at Manhattan Associates, where we dealt with messaging pipelines, latency, and partial failures in production. In those environments, we often made practical consistency versus availability decisions, especially when debugging data inconsistencies or throughput bottlenecks. However, this chapter strengthened my understanding of the theoretical foundation behind those tradeoffs and connected my practical experience to the deeper system model assumptions that make such limitations unavoidable.

week_5_readings

Edit 1 19 view

1 Followup Discussions

Resolved @585_f1

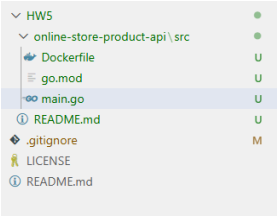
Wilson Neira 3 days ago

Hello Nithish Kadam Ganesh, I like how you explained that distributed systems are about choosing the right assumptions and tradeoffs, not just writing code, and it connects real-world experience with the deeper theory behind why things like CAP and failures are unavoidable.

Part II: Overview of the Start of our (Very!) Simple Online Store

Implementation

1. Implement GET and POST API




```go
// Routes supported:
// GET  /products/{productId}
// POST /products/{productId}/details
func parseProductID(path string) (id int32, ok bool, isProduc
    trim := strings.Trim(path, "/")
    parts := strings.Split(trim, "/")
```

2. Run with Docker

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\online-store-product-api\src> docker build -t productapi:local .
 => => extracting sha256:c65bb0c25578bf8f2a8b87d1996dfd93a5330c03195c8f0401cf88b2e0de9210                                    1.3s
 => => extracting sha256:13724d053e3111cb832ee8e49539a875afa879595fa76974c70b1981f057a675                                    0.5s
 => [internal] load build context                                                                                             0.2s
 => => transferring context: 6.84kB                                                                                           0.0s
 => [build 2/5] WORKDIR /app                                                                                                  1.2s
 => [build 3/5] COPY go.mod ./                                                                                                0.2s
 => [build 4/5] COPY main.go ./                                                                                               0.2s
 => [build 5/5] RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o server .                                               25.7s
 => [stage-1 2/3] COPY --from=build /app/server /server                                                                       0.2s
 => exporting to image                                                                                                        2.1s
 => => exporting layers                                                                                                       1.4s
 => => exporting manifest sha256:063fb3969bab82a4cc40224ec575a0bb4b1c842eecfb70f048a3cfb0885efef2                             0.1s
 => => exporting config sha256:add16abcb9ba3c30766a5eaf196cdf10415be70e9d34d444be9ac937dc7daa40                               0.0s
 => => exporting attestation manifest sha256:dbfc53aa1c20726f58c7911e9f38986d7e5649f8cc43942bfede2e136b276e6e                 0.1s
 => => exporting manifest list sha256:80456543d50732628725cba77b5dafb4ac3d4c2aefdbf1e57480fd7d6add8419                        0.0s
 => => naming to docker.io/library/productapi:local                                                                           0.0s
 => => unpacking to docker.io/library/productapi:local                                                                        0.2s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lq14x8kcvyk0gh1g5r5786s8k
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\online-store-product-api\src> docker run --rm -p 8080:8080 productapi
:local
2026/02/15 05:52:19 listening on :8080
```

3. Postman Create Variable

Overview | Development | Local | POST

Local

Variable	Value
baseUrl	http://localhost:8080
Add variable	

4. 204 - Product details added successfully (POST)

New Collection / {{baseUrl}}/products/12345/details

Save | Share

POST {{baseUrl}}/products/12345/details Send

Docs | Params | Authorization | Headers (9) | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2   "product_id": 12345,
3   "sku": "ABC-123-XYZ",
4   "manufacturer": "Acme Corporation",
5   "category_id": 456,
6   "weight": 1250,
7   "some_other_id": 789
```

Body | Cookies | Headers (1) | Test Results | 204 No Content | 101 ms | 64 B | Save Response

Raw | Preview | Visualize

```
1
```

5. 200 - Product found successfully (GET)

New Collection / **New Request**

GET {{baseUrl}} /products/12345

Docs Params Authorization Headers (7) Body Scripts Tests Settings

Headers 6 hidden

	Key	Value	Description
☐	Content-Type	application/json	
	Key	Value	Description

Body Cookies Headers (3) Test Results 200 OK 69 ms 236 B

{} JSON Preview Visualize


```
{
    "product_id": 12345,
    "sku": "ABC-123-XYZ",
    "manufacturer": "Acme Corporation",
    "category_id": 456,
    "weight": 1250,
    "some_other_id": 789
}
```

6. 404 - Product not found (GET)

New Collection / **404**

GET {{baseUrl}} /products/99999

Docs Params Authorization Headers (7) Body Scripts Tests Settings

Headers 6 hidden

	Key	Value	Desc
☐	Content-Type	application/json	
	Key	Value	Desc

Body Cookies Headers (3) Test Results 404 Not Found 25 ms

{} JSON Preview Debug with AI

```
1 {
2     "error": "PRODUCT_NOT_FOUND",
3     "message": "Product not found",
4     "details": "No product with id=99999"
5 }
```

7. 400 - Invalid input data (POST)

New Collection / **400**

POST {{baseUrl}} /products/12345/details

Docs Params Authorization Headers (9) Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSO

```
1 {
2   "product_id": 0,
3   "sku": "",
4   "manufacturer": "",
5   "category_id": 0,
6   "weight": -1,
7   "some_other_id": 0
8 }
```

Body Cookies Headers (3) Test Results 400 Bad Request 28 ms

{} JSON Preview Debug with AI

```
1 {
2     "error": "INVALID_INPUT",
3     "message": "The provided input data is invalid",
4     "details": "product_id must be a positive integer"
5 }
```

8. 500 - Internal server error

```go
105 func (s *productStore) handleGetProduct(w http.ResponseWriter
106     panic("boom")
107
108     id, ok, isProductRoute := parseProductID(r.URL.Path)
109     if !isProductRoute {
110         writeJSON(w, http.StatusNotFound, ErrorResponse{
```

docker build --no-cache -t productapi:local .
docker run --rm -p 8080:8080 productapi:local

New Collection / **200**

GET {{baseUrl}} /products/12345

Docs Params Authorization Headers (7) Body Scripts Tests Settings

Headers 6 hidden

	Key	Value	Des
☐	Content-Type	application/json	
	Key	Value	Des

Body Cookies Headers (3) Test Results 500 Internal Server Error 22 ms

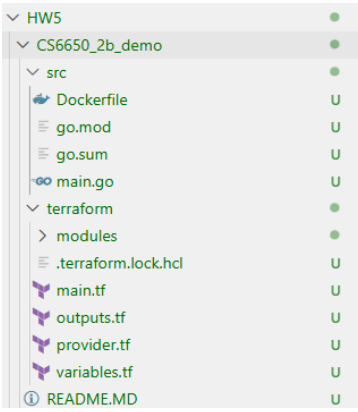
{} JSON Preview Debug with AI

```
1 {
2     "error": "INTERNAL_ERROR",
3     "message": "Internal server error"
4 }
```

Part III: Overview of Our Setup!

Infrastructure

1. Fork the infrastructure at this repo [https://github.com/RuidiH/CS6650_2b_demo]



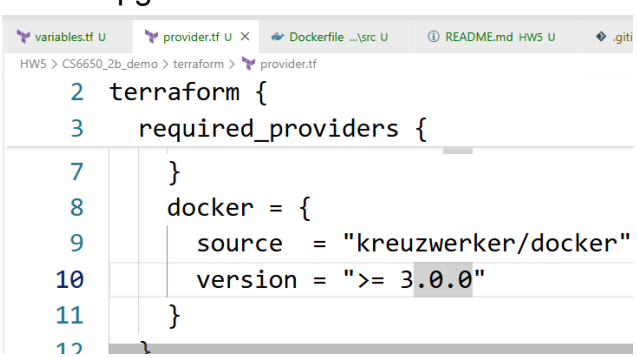

```
  default = "us-east-1"
}

# ECR & ECS settings
variable "ecr_repository_name" {
  type    = string
  default = "ecr_service"
}
```

2. aws configure

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\CS6650_2b_demo> aws configure
AWS Access Key ID [****************SLST]: ASIAUTE5J5TGB3XTS6MH
AWS Secret Access Key [****************gWpm]: 3FKMWIsuWpQieRwmvKtu6H+r6Mbe7mJCEHySrpBQ
AWS Session Token [****************kwvt]: IQoJb3JpZ2luX2VjEFoaCXVzLXdlc3QtMiJHMEUCIQDv0S/tXcv7NBAKlOsvxARD5Tpd7gxFwkYtSKgkc0F8DgIgeiuf79cc/KM
TE2R+npYIbXPbX0tEvdpa96Ut0uExL30qsgIIIxABGgwzMTYwMDk5OTk1NjQiDGFyeDucDBa7bKw60SqPAnsCb+5gDJE9q1YBTxILU0ZxEl3t9sVyqCwcwZ/Io0RjSZy+a0MBEaS6zzu/
CJ16xg1YhmnbrId/kAxt9KNVuaH2vBmEXGT1y6q9AkLwndoImZF+H+Ax/LgT09JC3cxsl6CzKqcaWYMMBEwBGs4A21PhI7isaiFbwdldWKKzItN65iOTi/ThfVIlk32T0LcrBgCvqdnsk
A81/YS/JEyCrlRlafKtkvxdJdQLKu7Ji4KpSwxbp5RRIMkjafWTjA9ptRI1Zwoq5QsCFEXU46ymcTeqAe9vPw7pw2N044B3kgeXKo6ODeqWIvcqiuHb8evcnD4JUq1LfGk9q+6MCOkfoQ
Voj2DS6bl/P57uf4ooffMwypLIzAY6nQHenqr2upQDuZNURXh5dXT9MFGaGREFScGicUTMJ/4YI12Sjd0hQxEq9ZuA4opOBbz2xlRq3/5nAdbsvLcm2JimVdyo37Ts1pofcoDBez7dCQf
5KnA5E/00Fn7Soj4keZO8MhcAmQyrNsTYCi92Aknn8v0ikW73vz6hs07zRD/4bN4peaBn47toagl2LqqOd5VRlLLsjOWIolXmT8+V
Default region name [us-east-1]:
Default output format [json]:
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\CS6650_2b_demo>
```

3. Upgrade Docker Provider Version




```
terraform {
  required_providers {
    }
    docker = {
      source  = "kreuzwerker/docker"
      version = ">= 3.0.0"
    }
  }
```

4. Apply Infrastructure

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\CS6650_2b_demo\terraform> terraform apply -auto-approve
docker_image.app: Still creating... [00m50s elapsed]
docker_image.app: Still creating... [01m00s elapsed]
docker_image.app: Still creating... [01m10s elapsed]
docker_image.app: Still creating... [01m20s elapsed]
docker_image.app: Still creating... [01m30s elapsed]
docker_image.app: Creation complete after 1m33s [id=sha256:a3f7f937210447c0f42d03f2c64db0610a8ef0d4990206a9f45913abb00f0c61316009999564.dkr.e
cr.us-east-1.amazonaws.com/ecr_service:latest]
docker_registry_image.app: Creating...
docker_registry_image.app: Creation complete after 5s [id=sha256:a3f7f937210447c0f42d03f2c64db0610a8ef0d4990206a9f45913abb00f0c61]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

ecs_cluster_name = "CS6650L2-cluster"
ecs_service_name = "CS6650L2"
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\CS6650_2b_demo\terraform>
```

5. Get public IP address of the ECS task

```
Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW5/CS6650_2b_demo/terraform (main)
$ aws ec2 describe-network-interfaces \
--network-interface-ids $(
    aws ecs describe-tasks \
    --cluster $(terraform output -raw ecs_cluster_name) \
    --tasks $(
        aws ecs list-tasks \
        --cluster $(terraform output -raw ecs_cluster_name) \
        --service-name $(terraform output -raw ecs_service_name) \
        --query 'taskArns[0]' --output text
    ) \
    --query "tasks[0].attachments[0].details[?name=='networkInterfaceId'].value" \
    --output text
) \
--query 'NetworkInterfaces[0].Association.PublicIp' \
--output text
44.204.248.66
```

44.204.248.66

6. Send some request

curl http://44.204.248.66:8080/albums

```
Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems/HW5/CS6650_2b_demo/terraform (main)
$ curl http://44.204.248.66:8080/albums
[
    {
        "id": "1",
        "title": "Blue Train",
        "artist": "John Coltrane",
        "price": 56.99
    },
    {
        "id": "2",
        "title": "Jeru",
        "artist": "Gerry Mulligan",
        "price": 17.99
    },
    {
        "id": "3",
        "title": "Sarah Vaughan and Clifford Brown",
        "artist": "Sarah Vaughan",
        "price": 39.99
    }
]
```

Part IV: Testing (yes this looks familiar!)

1. Create a locustfile.py for my endpoints

```python
class AlbumUser(HttpUser):
    wait_time = between(0.01, 0.05)

    @task(9)
    def list_albums(self):
        self.client.get("/albums")
```

2. Run Locust against AWS

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW5\CS6650_2b_demo> locust -f tests/locustfile.py --host "http://4
4.204.248.66:8080"
[2026-02-15 16:14:40,420] DESKTOP-5FCT03U/INFO/locust.main: Starting Locust 2.43.3
[2026-02-15 16:14:40,421] DESKTOP-5FCT03U/INFO/locust.main: Starting web interface at http://localhost:8089, press enter to open your default
 browser.
```

3. 50 users, 5 spawn rate, run ~1 minute

Start new load test

Number of users (peak concurrency) *
50

Ramp up (users started/second) *
5

Host
http://44.204.248.66:8080

Advanced options

START

LOCUST

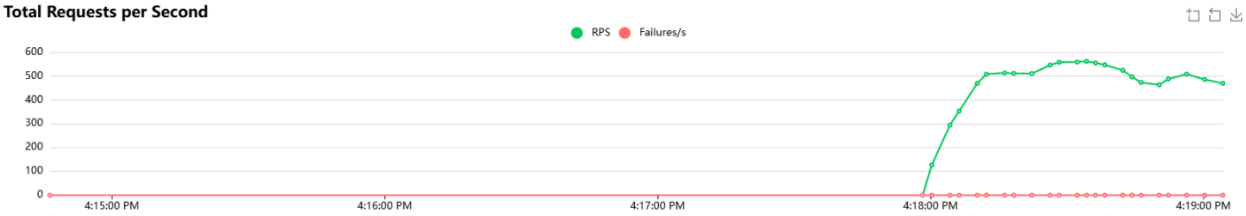
Host	Status	RPS	Failures
http://44.204.248.66:8080	STOPPED	469.6	0%

NEW RESET

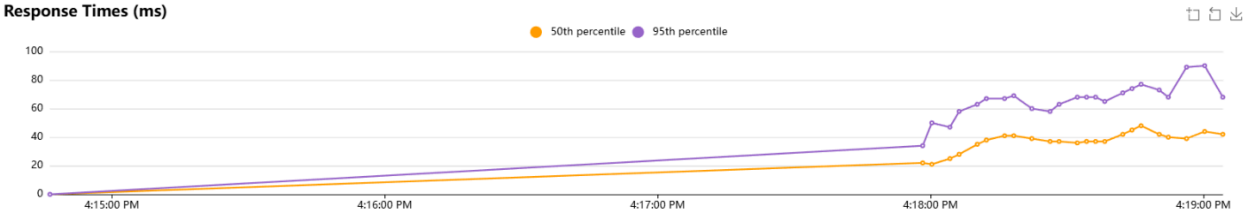
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	30298	0	39	69	97	42.1	11	521	382	423.2	0
GET	/albums/1	1093	0	39	71	96	42.79	14	503	95	14.9	0
GET	/albums/2	1100	0	39	70	97	42.67	15	519	90	16.2	0
GET	/albums/3	1072	0	39	67	100	41.45	12	189	117	15.3	0
	Aggregated	33563	0	39	69	97	42.12	11	521	354.62	469.6	0

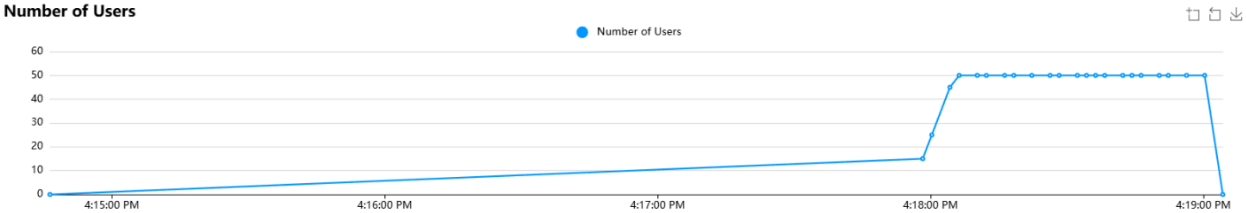
Total Requests per Second



Response Times (ms)



Number of Users



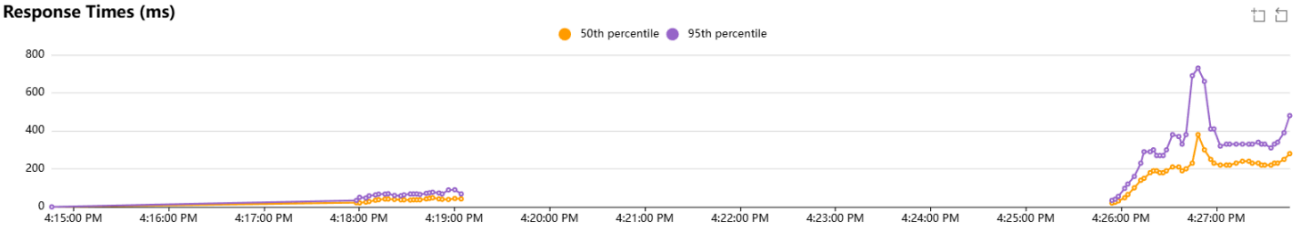
4. 200 users, 10 spawn rate, run ~1 minute

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA ! LOGS

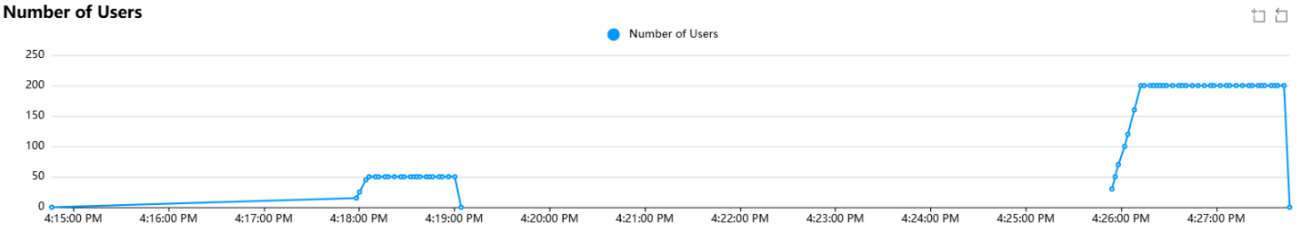
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	46709	0	210	370	580	213.54	12	1032	382	402.8	0
GET	/albums/1	1804	0	210	380	610	213.69	13	851	95	13.9	0
GET	/albums/2	1687	0	210	400	560	214.69	13	1022	90	15.7	0
GET	/albums/3	1769	0	210	370	620	213.61	13	885	117	14.9	0
	Aggregated	51969	0	210	370	580	213.58	12	1032	353.54	447.3	0

Total Requests per Second

Response Times (ms)



Number of Users

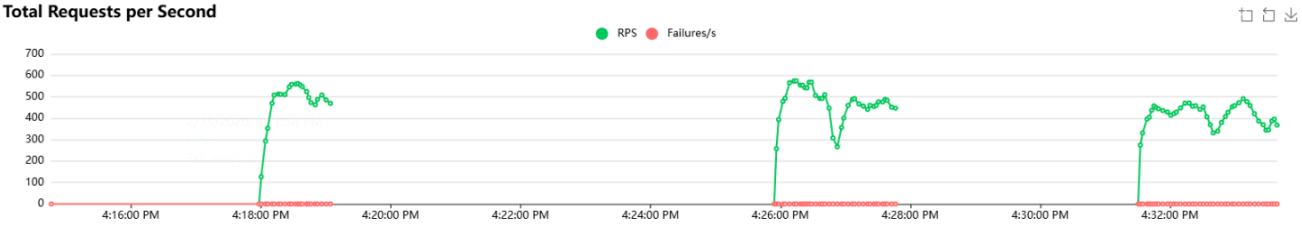


5. 500 users, 20 spawn rate, run ~1 minute

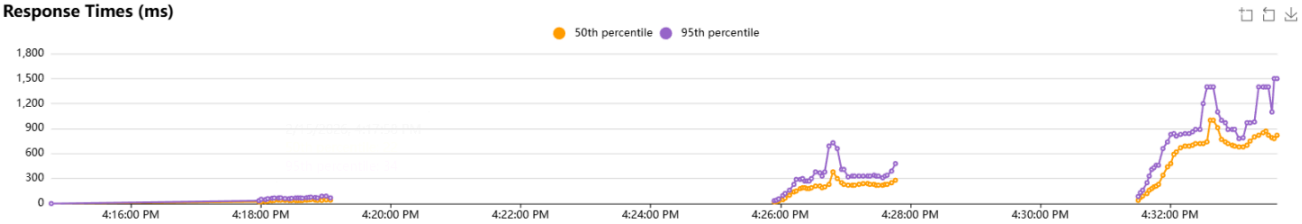
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA ! LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	48646	0	690	1100	1500	672.77	13	8473	382	330.4	0
GET	/albums/1	1763	0	690	1100	1500	671.67	21	8293	95	12.2	0
GET	/albums/2	1880	0	690	1100	1500	677.37	18	8762	90	12.7	0
GET	/albums/3	1874	0	690	1100	1400	668.4	17	6728	117	12.9	0
	Aggregated	54163	0	690	1100	1500	672.74	13	8762	353.35	368.2	0

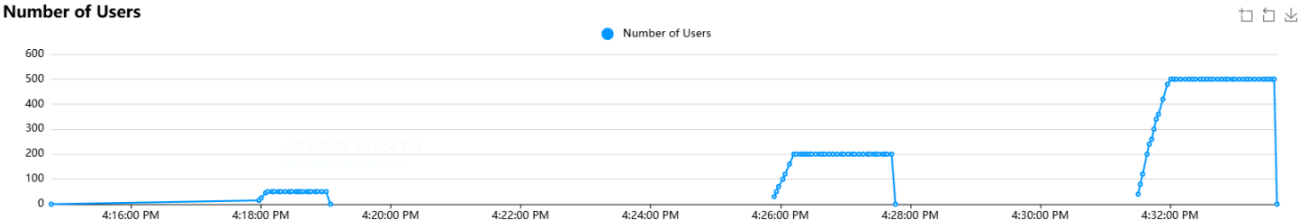
Total Requests per Second



Response Times (ms)



Number of Users



Which operations will be most common in real world scenario?

In a real store, reads (GETs like browsing products) likely happen way more often than writes (POST/updates).

How does that impact the data structure you are using to save your data?

Since reads dominate, you want something optimized for fast lookups like a hashmap (and later a cache) instead of slower scan-based structures.

Last week many people did not see any difference between HttpUser and FastHttpUser, why would that have been?

Because the test is usually limited by network/server latency (and the server's own speed), so the small client-side speedup from FastHttpUser may not noticeably change results.

WOW that api.yaml file has a LOT of stuff in it! How would you design (just design, not implement!) a scalable backend to handle the online store defined in that API?
I'd split it into separate services (products, users, orders, payments, shipping), put an API gateway in front, use a database per service plus caching, and scale each service independently behind load balancers.
Also, the official documention says "Terraform is a declarative language". What does that mean, and how is it different from imperative language?
Declarative means I describe the final infrastructure I want and Terraform figures out the steps, unlike imperative where I write the exact step-by-step commands to create everything.
How does that help us?
It makes deployments repeatable and safer because Terraform can plan changes, apply them consistently, and manage updates without me manually clicks around or forgetting steps.

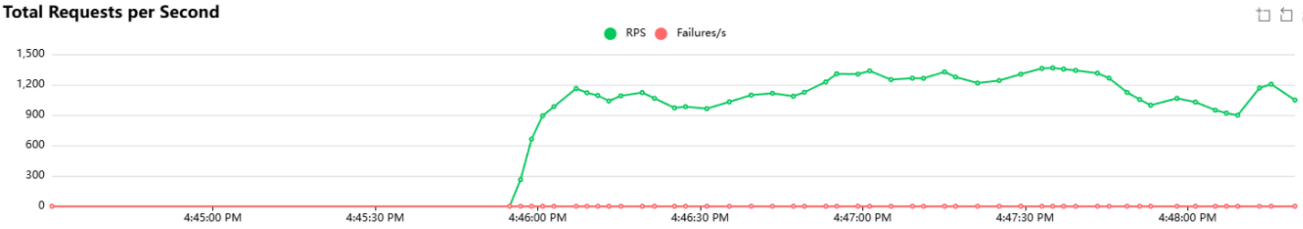
6. FastHttpUser, 500 users, 20 spawn rate, run ~1 minute

locust -f tests/locustfile_fast.py --host "http://44.204.248.66:8080"

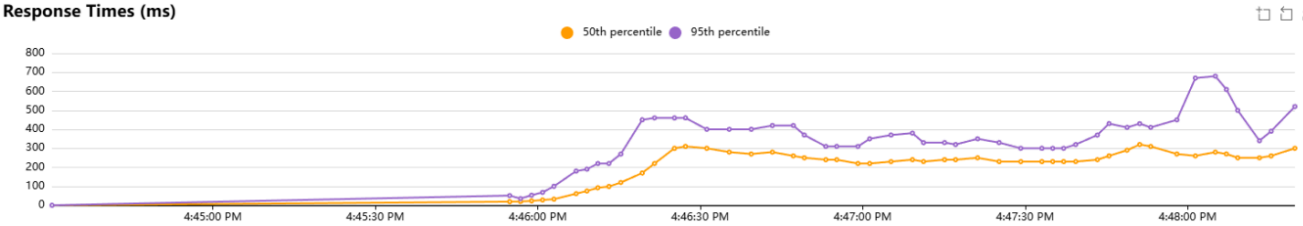
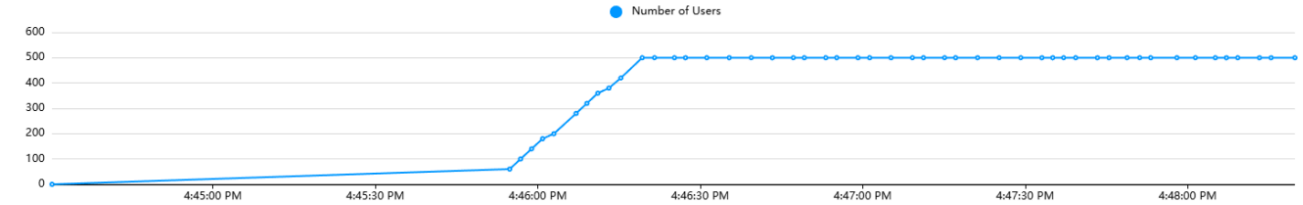
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA ! LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/albums	150669	0	240	400	580	243.13	10	2667	382	944.1	0
GET	/albums/1	5479	0	240	410	590	243.61	11	2664	95	33.5	0
GET	/albums/2	5648	0	240	410	580	243.44	11	2102	90	36.5	0
GET	/albums/3	5554	0	240	400	560	240.63	12	1537	117	34.1	0
	Aggregated	167350	0	240	400	580	243.07	10	2667	353.95	1048.2	0

Total Requests per Second



Response Times (ms)



Number of Users